Documentation — AI Study Planner

Vue d'ensemble

AI Study Planner est une application web qui permet à un utilisateur de :

- Gérer un emploi du temps hebdomadaire sous forme de calendrier
- Créer des objectifs d'apprentissage
- Générer automatiquement un plan d'étude par semaine grâce à une IA

L'application est divisée en **deux parties** qui tournent en parallèle :

```
graph LR
    A["🖥️ Navigateur\n(Frontend — port 5173)"] -->|"fetch POST /api/generate-plan"| B["⚙️
Serveur Node.js\n(Backend — port 3001)"]
    B -->|"Requête HTTP + clé API"| C["🤖 API Groq / Llama"]
    C -->|"Réponse JSON (plan)"| B
    B -->|"{ plan }"| A
```

Structure des fichiers

```
study-planner/
├── src/                  ← Code du frontend (ce que voit le navigateur)
│   ├── App.tsx           ← Tout le code React de l'appli
│   ├── MonApp.css        ← Tous les styles visuels
│   ├── main.tsx          ← Point d'entrée : lance React dans le HTML
│   └── index.css         ← Reset CSS de base
│
├── server/
│   └── index.ts          ← Serveur backend Express (appel IA)
│
├── .env                  ← Clé API (ne jamais mettre sur GitHub)
├── vite.config.js        ← Configuration de l'outil de dev Vite
└── package.json          ← Liste des dépendances npm
```

Fichier 1 — `src/main.tsx` (point d'entrée)

Rôle : C'est le fichier qui démarre l'application. Il dit à React de "prendre le contrôle" d'un élément HTML (`<div id="root">` dans `index.html`) et d'y afficher le composant principal `App` .

```
createRoot(rootElement).render(<App />)
```

On n'a pas besoin de le modifier. Il fait juste le lancement.

Fichier 2 — `src/App.tsx` (cœur du frontend)

Rôle : Contient **toute la logique et l'interface** de l'application sous forme de composants React.

Les composants

Un **composant** React est une fonction qui retourne du HTML (JSX/TSX). L'appli est découpée en plusieurs composants :

```
graph TD
    A["AIStudyPlanner\n(composant racine)"] --> B["ScheduleView\n(calendrier)"]
    A --> C["GoalsView\n(objectifs)"]
    A --> D["SettingsModal\n(paramètres)"]
    C --> E["GoalForm\n(formulaire objectif)"]
    C --> F["PlanModal\n(affichage du plan IA)"]
```

Les données (états)

Les **états** (`useState`) sont les données mémorisées par React. Quand un état change, React re-affiche automatiquement l'écran.

Variable d'état	Type	Rôle
`events`	`CalendarEvent[]`	Tous les événements du calendrier
`goals`	`Goal[]`	Tous les objectifs d'apprentissage
`settings`	`Settings`	Heure de début/fin du calendrier
`view`	`"schedule" | "goals"`	Onglet actif (emploi du temps ou objectifs)
`saveStatus`	`string`	Message "Sauvegardé ✓" affiché brièvement
`showSettings`	`boolean`	Si la modale de paramètres est ouverte

Les types TypeScript

Les interfaces définissent la forme exacte des données :

Interface	Champs principaux
`CalendarEvent`	`id`, `title`, `day`, `startTime`, `endTime`, `color`
`Goal`	`id`, `title`, `level`, `hoursPerWeek`, `learningType`, `plan?`
`StudyPlan`	`overview`, `weeks[]`, `milestones[]`, `tips[]`
`StudyWeek`	`weekNumber`, `theme`, `objectives[]`, `tasks[]`
`Settings`	`startHour`, `endHour`, `slotMinutes`

La sauvegarde automatique

```
sequenceDiagram
    participant U as Utilisateur
    participant R as React (useState)
    participant S as window.storage

    U->>R: Crée un événement
    R->>R: setEvents([...events, nouvel événement])
    R->>S: useEffect → saveData("events", events)
    S-->>R: Données sauvegardées
    R-->>U: Affiche "Sauvegardé ✓"
```

Au chargement de la page, `useEffect(load, [])` lit les données depuis `window.storage` et les recharge dans les états.

Le calendrier et l'algorithme rowSpan

Le calendrier est un **tableau HTML** (`<table>`). Chaque ligne = un créneau horaire. Chaque colonne = un jour de la semaine.

Problème : un événement de 9h00 à 11h00 doit visuellement occuper 2 lignes. On utilise l'attribut HTML `rowSpan` pour "fusionner" des cellules verticalement.

Comment ça marche :

```
Créneau  | Lundi       | Mardi | ...
---------|-------------|-------|----
09:00    | [Maths     ]|       |
         | [rowSpan=2 ]|       |       ← la cellule 09h occupe 2 lignes
10:00    |  (skippée) |       |       ← on ne génère pas cette cellule
11:00    |             |       |
```

Algorithme :

1. Pour chaque événement, on calcule `span = (endMinutes - startMinutes) / 60`
2. On stocke `rowSpanMap["Lundi-3"] = 2` (3 = index du slot 09h00)
3. On ajoute les cellules suivantes dans `coveredCells` → elles ne sont pas rendues
4. La cellule de départ reçoit `rowSpan={span}` et son badge remplit 100% de la hauteur

```
timeToMinutes("10:30") → 10*60 + 30 = 630
timeToMinutes("09:00") → 9*60 + 0  = 540
span = ceil((630 - 540) / 60) = ceil(1.5) = 2 lignes
```

Fichier 3 — `server/index.ts` (serveur backend)

Rôle : Mini serveur Express qui sert d'intermédiaire entre le frontend et l'API Groq (IA). Il garde la clé API côté serveur pour ne pas l'exposer dans le navigateur.

Flux complet d'une génération de plan

```
sequenceDiagram
    participant F as Frontend (App.tsx)
    participant B as Backend (server/index.ts)
    participant G as API Groq (Llama 3)

    F->>B: POST /api/generate-plan\n{ title, level, hoursPerWeek }
    B->>B: Construit le prompt texte
    B->>G: POST api.groq.com/v1/chat/completions\nHeader: Authorization: Bearer GROQ_API_KEY
    G-->>B: { choices[0].message.content: "{ ... JSON plan ... }" }
    B->>B: JSON.parse(content)
    B-->>F: { plan: { overview, weeks, milestones, tips } }
    F->>F: setGoals(...) → le plan s'affiche
```

La structure du prompt

Le prompt est le message envoyé à l'IA. On lui demande de répondre **uniquement en JSON** pour pouvoir le parser facilement :

```
Crée un plan d'étude pour : "React" (Niveau: beginner, 5h/semaine).
Réponds UNIQUEMENT en JSON valide.
```

```
Structure : { "overview": {...}, "weeks": [...], "milestones": [...], "tips": [...] }
```

La réponse de l'IA (structure JSON attendue)

Clé	Contenu
`overview.approach`	Texte décrivant la stratégie globale
`overview.totalWeeks`	Nombre total de semaines
`weeks[i].theme`	Thème de la semaine
`weeks[i].tasks[j].title`	Nom d'une tâche
`weeks[i].tasks[j].estimatedHours`	Durée estimée en heures
`milestones[i].title`	Nom d'un jalon important
`tips[i]`	Un conseil pratique

Fichier 4 — `vite.config.js` (configuration de l'outil de dev)

Rôle : Configure Vite, l'outil qui compile le code TypeScript/TSX en JavaScript et lance le serveur de développement.

Point important — le proxy :

```
proxy: { '/api': { target: 'http://localhost:3001' } }
```

Quand le frontend fait `fetch("/api/generate-plan")`, Vite redirige automatiquement la requête vers `http://localhost:3001/api/generate-plan`. Cela évite les erreurs CORS (le navigateur qui refuse les requêtes vers un autre port).

Fichier 5 — `src/MonApp.css` (styles)

Rôle : Définit l'apparence de tous les éléments visuels. Organisé en sections avec des commentaires.

Section	Classes principales
Variables de couleur	`:root` (--mint, --coral, --bg...)
Mise en page	`.app`, `.container`
En-tête	`.header`, `.header-title`
Onglets	`.tabs`, `.tab-btn`
Boutons	`.btn`, `.btn-primary`, `.btn-ghost`
Modales	`.modal-overlay`, `.modal-box`
Calendrier	`.calendar-table`, `.cal-td-cell`, `.event-badge`
Objectifs	`.goal-card`, `.goals-grid`

Fichier 6 — `.env` (variables d'environnement)

Rôle : Stocke les informations sensibles (clé API) séparément du code.

```
GROQ_API_KEY=gsk_xxxxxxxxxxxxxxxx
```

> ⚠️ *Ce fichier est dans `.gitignore` : il ne sera jamais envoyé sur GitHub. C'est essentiel pour ne pas exposer la clé publiquement.*

Fichier 7 — `package.json` (liste des dépendances)

Rôle : Liste toutes les bibliothèques utilisées et les commandes disponibles.

Commande	Ce qu'elle fait
`npm run dev`	Lance le frontend sur http://localhost:5173
`npm run server`	Lance le backend sur http://localhost:3001
`npm run build`	Compile le projet pour la mise en production

Dépendance	Rôle
`react`, `react-dom`	Bibliothèque UI frontend
`express`	Serveur backend Node.js
`cors`	Autorise les requêtes entre frontend et backend
`dotenv`	Lit le fichier `.env`
`tsx`	Exécute les fichiers TypeScript directement
`vite`	Outil de compilation et serveur de dev